

Padrão de Commits do Projeto

Guia de uso do Conventional Commits para o repositório do sistema de Gerenciamento de Empresas

1. Objetivo

Este documento define o padrão que todos os membros do grupo devem seguir ao escrever mensagens de commit no repositório. O objetivo é manter um histórico de alterações claro, organizado e fácil de entender, permitindo identificar rapidamente o tipo e o propósito de cada mudança feita no código.

2. Formato da mensagem

Toda mensagem de commit deve seguir a estrutura abaixo:

tipo: descrição curta e objetiva da alteração

O **tipo** indica a natureza da mudança (feat, fix, docs, etc.) e a **descrição** deve estar no infinitivo/imperativo, em letras minúsculas, sem ponto final no fim da frase.

3. Tipos de commit e quando usar cada um

Tipo	Quando usar	Exemplo
feat	Ao adicionar uma nova funcionalidade ao sistema.	feat: add dark launch support for new billing flow
fix	Ao corrigir um bug ou comportamento incorreto no código.	fix: prevent null pointer in customer repository
refactor	Ao reestruturar código existente sem alterar seu comportamento externo (sem adicionar funcionalidade nem corrigir bug).	refactor: extract payment validation service
docs	Ao criar ou atualizar documentação (README, comentários de documentação, arquivos em /docs).	docs: update migration strategy documentation
style	Ao alterar formatação, indentação, espaçamento ou nomes, sem mudar a lógica do código.	style: simplify conditional formatting in reports
test	Ao adicionar, corrigir ou atualizar testes automatizados.	test: add integration test for login endpoint
perf	Ao fazer uma alteração cujo objetivo principal é melhorar performance.	perf: optimize cache lookup for session tokens
build	Ao alterar o sistema de build ou dependências externas do projeto.	build: upgrade spring boot dependencies
chore	Tarefas de manutenção que não afetam o código de produção nem os testes (ex.: configs, limpeza de arquivos).	chore: remove deprecated feature flags
revert	Ao reverter um commit anterior.	revert: rollback unstable notification service

4. Boas práticas

- Escrever a descrição em inglês, seguindo o padrão utilizado na comunidade open source (opcional, mas recomendado).
- Manter a descrição curta (idealmente até 72 caracteres) e objetiva.
- Fazer um commit por alteração lógica, evitando misturar tipos diferentes (ex.: um fix junto com um feat) no mesmo commit.
- Usar o corpo do commit (linha em branco após a descrição) quando for necessário explicar o motivo ou detalhes adicionais da mudança.
- Revisar o commit antes de enviar (git commit --amend) caso perceba um erro de digitação ou tipo incorreto.

5. Exemplos completos

```
git commit -m "feat: add dark launch support for new billing flow"
git commit -m "fix: prevent null pointer in customer repository"
git commit -m "refactor: extract payment validation service"
git commit -m "docs: update migration strategy documentation"
git commit -m "style: simplify conditional formatting in reports"
git commit -m "test: add integration test for login endpoint"
git commit -m "perf: optimize cache lookup for session tokens"
git commit -m "build: upgrade spring boot dependencies"
git commit -m "chore: remove deprecated feature flags"
git commit -m "fix: handle timeout in external api requests"
git commit -m "revert: rollback unstable notification service"
```

6. Onde este documento fica no repositório

```
docs/development/commit-conventions.pdf
```